3 CPU 微体系结构

本章简要概述了对软件性能产生直接影响的关键 CPU 微体系结构特性。本章的目标不是涵盖 CPU 架构的所有细节和权衡，这些内容已在文献中广泛讨论 [Hennessy & Patterson, 2017]。相反，本章快速回顾了现代处理器中存在的 CPU 硬件特性。

3.1 指令集架构

指令集是软件用来与硬件通信的词汇。指令集架构（ISA）定义了软件与硬件之间的约定。英特尔 x86、ARM v8 和 RISC-V 是当前广泛部署的 ISA 的示例。所有这些都是 64 位架构，即所有地址计算都使用 64 位。ISA 开发人员和 CPU 架构师通常确保符合规范的软件或固件将在使用规范构建的任何处理器上执行。广泛部署的 ISA 特许经营权通常还确保向后兼容性，使为处理器的 GenX 版本编写的代码将继续在 GenX+i 上执行。

大多数现代架构可以归类为通用寄存器为基础的，采用加载-存储架构，操作数明确指定，仅使用加载和存储指令访问内存。除了在 ISA 中提供基本功能（如加载、存储、控制和使用整数和浮点数进行标量算术运算）之外，广泛部署的架构继续增强其 ISA 以支持新的计算范式。这些包括增强的向量处理指令（例如，英特尔 AVX2、AVX512、ARM SVE）和矩阵/张量指令（英特尔 AMX）。映射到使用这些高级指令的软件通常在性能上提供数量级的改进。

现代 CPU 支持 32 位和 64 位精度的算术运算。随着机器学习和人工智能领域的快速发展，行业对于用于驱动显著性能改进的变量的替代数值格式重新产生了兴趣。研究表明，使用较少位数表示变量，既可以保持机器学习模型的性能，又可以节省计算和内存带宽。因此，几个 CPU 特许经营权最近已经在 ISA 中添加了对较低精度数据类型的支持，例如 8 位整数（int8，例如，英特尔 VNNI）、16 位浮点数（fp16，bf16），除了传统的 32 位和 64 位格式进行算术运算。

3.2 流水线技术

流水线是使 CPU 快速运行的基础技术，其中多个指令在它们的执行过程中重叠。CPU 中的流水线技术从汽车组装线中汲取灵感。指令的处理被分为多个阶段。这些阶段并行运行，同时处理不同指令的不同部分。DLX 是由 John L. Hennessy 和 David A. Patterson 于 1994 年设计的一个相对简单的架构。正如 [Hennessy & Patterson, 2017] 中定义的那样，它具有 5 级流水线，包括：

1. 指令获取（IF）
2. 指令解码（ID）
3. 执行（EXE）
4. 存储访问（MEM）
5. 写回（WB）

图 7 显示了 5 级流水线 CPU 的理想流水线视图。在周期 1 中，指令 x 进入流水线的 IF 阶段。在下一个周期中，随着指令 x 移动到 ID 阶段，程序中的下一条指令进入 IF 阶段，依此类推。一旦流水线满了，就像上面的周期 5 一样，CPU 的所有流水线阶段都在忙于处理不同的指令。没有流水线技术，指令`x+1`在指令`x`完成其工作之后才能开始执行。

现代高性能 CPU 具有多个流水线阶段，通常从 10 到 20 个或更多，具体取决于架构和设计目标。这涉及比之前介绍的简单 5 级流水线更复杂的设计。例如，解码阶段可能会分成几个新阶段，我们可能在执行阶段之前添加新阶段来缓冲解码指令，等等。

流水线 CPU 的吞吐量定义为单位时间内完成并退出流水线的指令数量。对于任何给定指令，其延迟是流水线各个阶段的总时间。由于流水线的所有阶段都彼此关联，因此每个阶段都必须准备好以锁步方式移动到下一条指令。将指令从一个阶段移动到下一个阶段所需的时间定义了 CPU 的基本机器周期或时钟。对于给定的流水线，所选择的时

Instruction	Clock cycle								
	1	2	3	4	5	6	7	8	9
Instruction x	IF	ID	EXE	MEM	WB				
Instruction x+1		IF	ID	EXE	MEM	WB			
Instruction x+2			IF	ID	EXE	MEM	WB		
Instruction x+3				IF	ID	EXE	MEM	WB	
Instruction x+4					IF	ID	EXE	MEM	WB

Figure 7: 简单的 5 级流水线图。

钟值由流水线中最慢的阶段定义。CPU 硬件设计人员努力平衡每个阶段可以完成的工作量，因为这直接定义了 CPU 的操作频率。增加频率可以提高性能，通常涉及平衡和重新设计流水线以消除由最慢的流水线阶段引起的瓶颈。

在一个理想的、完全平衡且不产生任何停顿的流水线中，流水线机器中每个指令的时间由以下公式给出：

$$\text{流水线机器中每条指令的时间} = \frac{\text{非流水线机器中每条指令的时间}}{\text{流水线阶段数}}$$

在实际实现中，流水线技术引入了几个限制，限制了上述理想模型。流水线冲突阻止了理想的流水线行为，导致停顿。流水线冲突分为三类：结构冲突、数据冲突和控制冲突。幸运的是，对于程序员来说，在现代 CPU 中，所有类别的冲突都由硬件处理。

- **结构冲突**：由资源冲突引起。在很大程度上，它们可以通过复制硬件资源来消除，例如使用多端口寄存器或存储器。然而，消除所有这些冲突可能在硅面积和功耗方面变得非常昂贵。
- **数据冲突**：由程序中的数据依赖性引起，分为三种类型：
  - 写后读（RAW）冲突需要依赖读取在写入之后执行。当指令`x+1`在先前的指令`x`写入源之前读取源时，就会发生这种情况，导致读取到错误的值。CPU 实现了从流水线的后期阶段向前期阶段传递数据的数据转发（称为“绕道”），以减轻与 RAW 冲突相关的惩罚。其思想是在指令`x`完全完成之前，可以将指令`x`的结果转发到指令`x+1`。如果我们看一个例子：

```
R1 = R0 ADD 1
R2 = R1 ADD 2
```

  对寄存器 R1 存在 RAW 依赖。如果我们直接在添加`R0 ADD 1`完成后（从`EXE`流水线阶段），取值，我们就不需要等到`WB`阶段完成，值就会被写入到寄存器文件。绕道有助于节省一些周期。流水线越长，绕道就越有效果。

  - 读后写（WAR）冲突需要依赖写入在读取之后执行。当指令`x+1`在指令`x`读取源之前写入源时，就会发生这种情况，导致读取到错误的新值。WAR 冲突不是真正的依赖关系，可以通过一种称为寄存器重命名[32]的技术来消除。这是一种从物理寄存器中抽象逻辑寄存器的技术。CPU 通过保留大量物理寄存器来支持寄存器重命名。逻辑寄存器（体系结构定义的寄存器）只是覆盖更广的寄存器文件上的别名。通过这种体系结构状态[33]的解耦，解决 WAR 冲突变得简单：我们只需要为写操作使用不同的物理寄存器。例如：

```
R1 = R0 ADD 1
R0 = R2 ADD 2
```

[32] 寄存器重命名 - https://en.wikipedia.org/wiki/Register_renaming。
[33] 体系结构状态 - https://en.wikipedia.org/wiki/Architectural_state。

对寄存器 R0 存在 WAR 依赖。由于我们有大量的物理寄存器，我们可以简单地为从写操作开始的所有R0寄存器的出现重新命名。一旦通过重新命名寄存器R0消除了 WAR 冲突，我们就可以以任何顺序安全地执行这两个操作。

  - 写后写（WAW）冲突需要依赖写入在写入之后执行。当指令x+1在指令x写入源之前写入源时，就会发生这种情况，导致写入的顺序错误。通过寄存器重命名，可以消除 WAW 冲突，允许两个写入以任何顺序执行，同时保持正确的最终结果。

- **控制冲突**：由程序流程的变化引起。它们起源于对分支和其他改变程序流程的指令进行流水线处理。决定分支方向（取或不取）的分支条件在执行流水线阶段解决。因此，除非消除控制冲突，否则无法对下一条指令的提取进行流水线处理。动态分支预测和在下一节中描述的推测执行等技术用于克服控制冲突。

3.3 开发指令级并行性（ILP）

程序中的大多数指令都适合进行流水线处理并并行执行，因为它们是独立的。现代 CPU 实现了大量额外的硬件功能来利用这种指令级并行性（ILP）。与先进的编译器技术配合使用，这些硬件功能可以提供显著的性能改进。

3.3.1 乱序执行（OOO Execution）

图 7 中的流水线示例显示所有指令按顺序通过流水线的不同阶段移动，即按照它们在程序中出现的顺序。大多数现代 CPU 支持乱序执行（OOO execution），即顺序指令可以以任意顺序进入执行流水线阶段，仅受其依赖关系的限制。乱序执行的 CPU 仍然必须产生与所有指令按程序顺序执行相同的结果。当指令最终执行且其结果正确且可见于体系结构状态时，该指令被称为 retired。为确保正确性，CPU 必须按照程序顺序使所有指令退役。乱序执行主要用于避免由于依赖关系引起的停顿而导致 CPU 资源的低利用率，特别是在下一节中描述的超标量引擎中。

这些指令的动态调度是由复杂的硬件结构（如分数板 (scoreboards)）和技术（如寄存器重命名）启用的，以减少数据冲突。在 1960 年代，一些支持动态调度和乱序执行的工作包括Tomasulo 算法[34]（在 IBM360 中实现）和Scoreboading[35]（在 CDC6600 中实现）。这些开创性的工作影响了所有现代 CPU 架构。分数板硬件用于调度按顺序退役和所有机器状态更新。它跟踪每条指令的数据依赖关系以及流水线中数据的可用位置。大多数实现都致力于在硬件成本与潜在回报之间取得平衡。通常，分数板的大小决定了硬件可以查找的独立指令的距离有多远以进行调度。

Instruction	Clock cycle									
	1	2	3	4	5	6	7	8	9	10
Instruction x	IF	ID	EXE	MEM	WB					
Instruction x+1		IF	ID			EXE	MEM	WB		
Instruction x+2			IF	ID	EXE	MEM			WB	
Instruction x+3				IF	ID		EXE	MEM		WB

Figure 8: 乱序执行的概念。

图 8 详细说明了乱序执行的概念，以一个示例为例。假设由于冲突，指令 x+1 在周期 4 和 5 无法执行。顺序 CPU 将阻塞所有后续指令进入 EXE 流水线阶段。在具有乱序执行的 CPU 中，不具有任何冲突（例如，指令x+2）的后续指令可以进入并完成其执行。所有指令仍然按顺序退役，即指令按程序顺序完成 WB 阶段。

[34] Tomasulo algorithm - https://en.wikipedia.org/wiki/Tomasulo_algorithm.

[35] Score boarding - https://en.wikipedia.org/wiki/Scoreboarding.

3.3.2 超标量引擎和 VLIW

大多数现代 CPU 都是超标量的，即它们可以在给定周期内发出多个指令。发出宽度是在同一周期内发出的指令的最大数量。当前一代 CPU 的典型发出宽度范围为 2 到 6。为了确保正确平衡，这种超标量引擎还具有多个执行单元和/或流水线执行单元。CPU 还将超标量功能与深度流水线和乱序执行结合起来，以提取给定软件的最大 ILP。

Instruction	Clock cycle					
	1	2	3	4	5	6
Instruction x	IF	ID	EXE	MEM	WB	
Instruction x+1	IF	ID	EXE	MEM	WB	
Instruction x+2		IF	ID	EXE	MEM	WB
Instruction x+3		IF	ID	EXE	MEM	WB

Figure 9: 简单 2 路超标量 CPU 的流水线图。

图 9 显示了支持 2 路发出宽度的 CPU 的示例，即在每个周期内，每个流水线阶段处理两条指令。超标量 CPU 通常支持多个独立执行单元，以确保指令在流水线中无冲突地流动。除了流水线化之外，复制执行单元还进一步提高了机器的性能。

像英特尔 Itanium 这样的体系结构将调度超标量、多执行单元机器的负担从硬件转移到了编译器，使用了一种称为 VLIW（Very Long Instruction Word）的技术。其理念是通过要求编译器选择正确的指令组合来使机器保持完全利用。编译器可以使用软件流水线和循环展开等技术，查看远远超出硬件结构合理支持范围的指令，以找到正确的 ILP。

3.3.3 推测执行

正如前一节所述，如果指令在分支条件解析之前被停顿，控制冲突可能会导致流水线的显著性能损失。为了避免这种性能损失，一种技术是使用硬件分支预测逻辑来预测分支的可能方向，并允许从预测路径执行指令（推测执行）。

让我们考虑 Listing 3.3.3 中的短代码示例。为了让处理器了解它应该执行哪个函数，它应该知道条件`a < b`是 false 还是 true。如果不知道，CPU 将等待分支指令的结果，如图 10 所示。

代码示例：推测执行

```
if (a < b)
  foo();
else
  bar();
```

Instruction	Clock cycle							
	1	2	3	4	5	6	7	8
BRANCH (a < b)	IF	ID	EXE	MEM	WB			
CALL foo				IF	ID	EXE	MEM	WB
// INSTR from foo					IF	ID	EXE	MEM

Figure 10: 无推测

Instruction	Clock cycle						
	1	2	3	4	5	6	7
BRANCH (a < b)	IF	ID	EXE	MEM	WB		
CALL foo		IF*	ID*	EXE	MEM	WB	
// INSTR from foo			IF*	ID	EXE	MEM	WB

Figure 11: 推测执行

使用推测执行，CPU 猜测分支的结果，并开始处理所选择路径的指令。假设处理器预测条件`a < b`将被评估为 true。它继续执行而不等待分支结果，并推测性地调用函数`foo`（见图 11 ，推测工作用*标记）。直到条件解析为止，机器状态变化才能被提交，以确保机器的体系结构状态永远不会受到推测执行指令的影响。在上面的例子中，分支指令比较了两个标量值，这很快。但实际上，分支指令可能取决于从内存加载的值，这可能需要数百个周期。如果预测是正确的，它将节省大量周期。然而，有时预测是错误的，应该调用函数`bar`。在这种情况下，推测执行的结果必须被取消并丢弃。这称为分支错误预测的惩罚，我们将在 Section 4.8中讨论。

为了跟踪推测的进展，现代 CPU 支持一种称为重新排序缓冲区（ROB）的结构。ROB 维护所有指令执行的状态，并按顺序退役指令。推测执行的结果写入 ROB，并按程序流程的相同顺序提交给体系结构寄存器，仅在推测是正确的情况下。CPU 还可以将推测执行与乱序执行相结合，并使用 ROB 来跟踪推测和乱序执行。

3.3.4 分支预测

正如我们刚才看到的，正确的预测极大地提高了执行效率，因为它们允许 CPU 在没有前一条指令结果的情况下继续前进。然而，错误的推测通常会导致昂贵的性能惩罚。现代 CPU 采用了复杂的动态分支预测机制，提供非常高的准确性，并能够适应分支行为的动态变化。有三种类型的分支可能需要以特殊方式处理：

- **无条件跳转和直接调用**：它们最容易预测，因为它们总是被执行并且每次都以相同的方向执行。
- **条件分支**：它们有两个潜在的结果：被执行或不被执行。被执行的分支可以向前或向后跳转。前向条件分支通常用于生成`if-else`语句，其不被执行的可能性很高，因为通常表示错误检查代码。后向条件跳转经常出现在循环中，并用于转到循环的下一次迭代；此类分支通常被执行。
- **间接调用和跳转**：它们具有许多目标。间接跳转或间接调用可以生成`switch`语句、函数指针或`virtual`函数。函数返回也值得关注，因为它也有许多潜在的目标。

大多数预测算法基于分支的先前结果。分支预测单元（BPU）的核心是分支目标缓冲区（BTB），它为每个分支缓存目标地址。预测算法每个周期都要查询 BTB，以生成下一个要提取指令的地址。CPU 使用该新地址提取下一个指令块。如果在当前提取块中没有识别出分支，则提取的下一个地址将是下一个顺序对齐的提取块（顺序提取）。

无条件分支不需要预测；我们只需要在 BTB 中查找目标地址。记住，每个周期 BPU 都需要生成下一个地址，以避免流水线停滞。我们可以仅从指令编码中提取地址，但这样我们必须等到解码阶段结束，这将在流水线中引入一个气泡并使事情变慢。因此，在提取分支时必须确定下一个提取地址。

对于条件分支，我们首先需要预测分支是否被执行。如果未执行，则我们会顺序执行，并且无需查找目标。否则，我们将在 BTB 中查找目标地址。条件分支通常占据总分支的最大部分，并且是生产软件中错误预测的主要来源。对于间接分支，我们需要选择可能的目标之一，但是预测算法可以与条件分支非常相似。

所有预测机制都试图利用两个重要原则，这与我们稍后将讨论的缓存类似：

- **时间相关性**：分支的解决方式可能是下次执行时解决方式的很好的预测器。这也称为局部相关性。
- **空间相关性**：几个相邻的分支可能以高度相关的方式解决（执行的首选路径）。这也称为全局相关性。

最佳的准确性通常通过同时利用局部和全局相关性来实现。因此，我们不仅查看当前分支的结果历史，还将其与其他分支的结果相关联。

另一种常见的技术是混合预测。其思想是一些分支具有偏向行为。例如，如果条件分支 99.9% 的时间都朝着一个方向走，就没有必要使用复杂的预测器并污染其数据结构。可以使用一个简单得多的机制。另一个示例是循环分支。如果分支具有循环行为，则可以使用专用的循环预测器进行预测，该预测器将记住循环通常执行的迭代次数。

如今，最先进的预测主要由类似 TAGE [Seznec & Michaud, 2006] 或感知器-based [Jimenez & Lin, 2001] 的预测器主导。冠军[36]分支预测器在每 1000 条指令中不到 3 次错误预测。现代 CPU 在大多数工作负载上通常达到超过 95% 的预测率。

3.4 SIMD 多处理器

另一种被广泛用于许多工作负载的多处理器变体被称为单指令多数据（SIMD）。顾名思义，在 SIMD 处理器中，单个指令在单个周期内使用多个独立的功能单元操作许多数据元素。向量和矩阵的操作很适合 SIMD 架构，因为向量或矩阵的每个元素都可以使用相同的指令进行处理。SIMD 架构可以更有效地处理大量数据，最适合涉及向量操作的数据并行应用程序。

图 12 展示了代码 Listing 3.4 中的标量和 SIMD 执行模式。在传统的 SISD（单指令，单数据）模式中，加法操作分别应用于数组 a 和 b 的每个元素。然而，在 SIMD 模式中，加法同时应用于多个元素。如果我们针对具有能够对 256 位向量执行操作的执行单元的 CPU 架构进行优化，我们可以使用单个指令处理四个双精度元素。这导致发出的指令数量减少了 4 倍，并且可能比四个标量计算获得 4 倍的加速。但是在实践中，由于各种原因，性能优势并不那么直接。

代码清单：SIMD 执行

```
double *a, *b, *c;
for (int i = 0; i < N; ++i) {
  c[i] = a[i] + b[i];
}
```

对于常规的 SISD 指令，处理器使用通用寄存器。同样，对于 SIMD 指令，CPU 有一组 SIMD 寄存器，用于保持从内存加载的数据和存储计算的中间结果。在我们的示例中，与数组 a 和 b 对应的两个连续的 256 位数据区域将从内存加载并存储在两个单独的向量寄存器中。接下来，将进行逐元素加法运算，并将结果存储在一个新的 256 位向量寄存器中。最后，将结果从向量寄存器写入对应于数组 c 的 256 位内存区域。请注意，数据元素可以是整数或浮点数。

大多数流行的 CPU 架构都具有向量指令，包括 x86、PowerPC、Arm 和 RISC-V。1996 年，英特尔推出了 MMX，一个针对多媒体应用程序设计的 SIMD 指令集。随后，英特尔引入了具有增强功能和增加向量大小的新指令集：SSE、AVX、AVX2、AVX-512。Arm 在其各个版本的架构中选择性地支持 128 位 NEON 指令集。在第 8 版（aarch64）中，这种支持变得是强制性的，并添加了新的指令。

随着新的指令集变得可用，开始着手使它们对软件工程师可用。利用 SIMD 指令所需的软件更改称为代码向量化。最初，SIMD 指令是用汇编语言编程的。后来，引入了特殊的编译器内置函数，它们是一对一映射到 SIMD 指令的小函数。今天，所有主要的编译器都支持针对流行处理器的自动向量化，即它们可以直接从 C/C++、Java、Rust 等高级语言编写的代码生成 SIMD 指令。

为了使代码能够在支持不同向量长度的系统上运行，Arm 引入了 SVE 指令集。其定义特征是可伸缩向量的概念：它们的长度在编译时是未知的。使用 SVE，无需将软件移植到每种可能的向量长度。用户不必重新编译其应用程序的源代码以利用在较新的 CPU 代中可用的更宽向量。可伸缩向量的另一个示例是 RISC-V V 扩展（RVV），该扩展于 2021 年底获得批准。一些实现支持相当宽（2048 位）的向量，并且最多可以将八个向量组合在一起，形成16,384位

[36] 第五届冠军分支预测大赛 - https://jilp.org/cbp2016

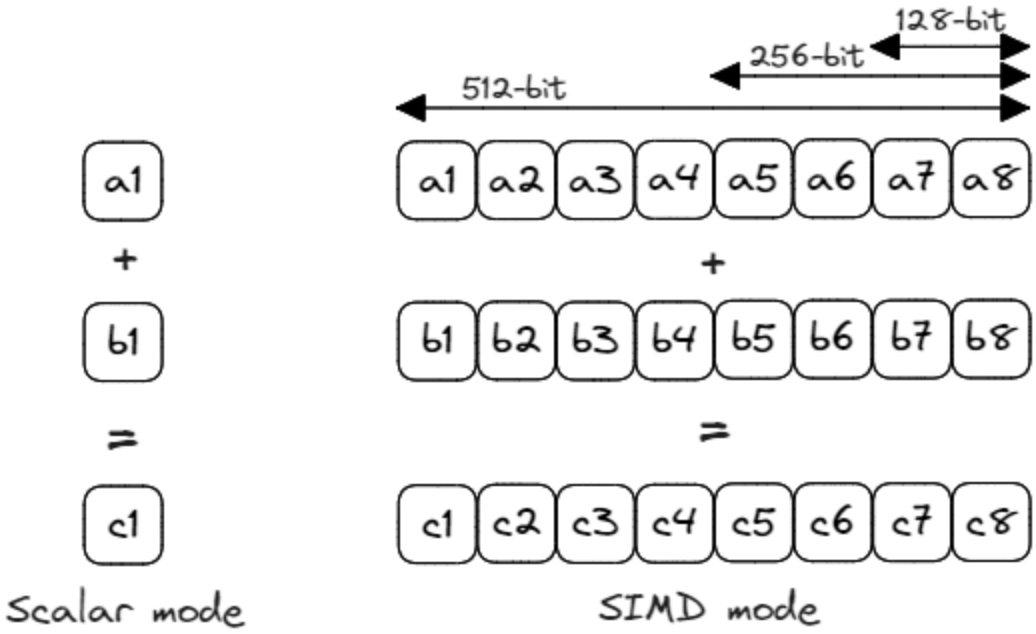


Figure 12: 标量和 SIMD 操作的示例。

的向量，这极大地减少了执行的指令数量。在每次循环迭代中，用户代码通常执行 `ptr += number_of_lanes`，其中 `number_of_lanes` 在编译时是未知的。ARM SVE 为这种长度相关操作提供了特殊指令，而 RVV 允许程序员查询/设置 `number_of_lanes`。

此外，CPU 越来越多地加速机器学习中经常使用的矩阵乘法。英特尔的 AMX 扩展，支持 Sapphire Rapids，将形状为 16x64 和 64x16 的 8 位矩阵相乘，并累积为 32 位的 16x16 矩阵。相比之下，苹果 CPU 中无关但同名的 AMX 扩展，以及 ARM 的 SME 扩展，计算存储在特殊的 512 位寄存器或可伸缩向量中的一行和一列的外积。

最初，SIMD 是由多媒体应用程序和科学计算推动的，但后来在许多其他领域找到了用途。随着时间的推移，SIMD 指令集中支持的操作集稳步增加。除了图 12 中显示的直接算术运算外，SIMD 的新用途还包括：

- 字符串处理：查找字符，验证 UTF-8[37]，解析 JSON[38]和 CSV[39]；
- 哈希运算[40]，随机生成[41]，密码学 (AES)；
- 列式数据库（位打包、过滤、连接）；
- 对内置类型进行排序（VQSort[42]，QuickSelect）；
- 机器学习和人工智能（加速 PyTorch、Tensorflow）。

3.5 开发线程级并行性

前面描述的技术依赖于程序中可用的并行性来加速执行。除此之外，CPU 还支持利用跨进程和/或线程的并行性的技术。接下来，我们将讨论三种利用线程级并行性（TLP）的技术：多核系统、同时多线程和混合架构。这些技术使得能够充分利用可用的硬件资源，并提高系统的吞吐量。

[37] UTF-8 验证 - https://github.com/rusticstuff/simdutf8
[38] 解析 JSON - https://github.com/simdjson/simdjson.
[39] 解析 CSV - https://github.com/geofflangdale/simdcsv
[40] SIMD 哈希运算 - https://github.com/google/highwayhash
[41] 随机生成 - Abseil 库
[42] 排序 - VQSort

3.5.1 多核系统

随着处理器架构开始达到半导体设计和制造的实际限制，GHz 竞赛减缓，设计师不得不专注于其他创新来提高 CPU 性能。其中一个关键方向是多核设计，试图增加每代的核心数。其想法是在单个芯片上复制多个处理器核心，并让它们同时为不同的程序提供服务。例如，一个核心可以运行 Web 浏览器，另一个核心可以渲染视频，另一个可以播放音乐，所有这些都可以同时进行。对于服务机器，来自不同客户端的请求可以在不同的核心上进行处理，这可以极大地增加系统的吞吐量。

第一个面向消费者的双核处理器是英特尔 Core 2 Duo，于 2005 年发布，随后是同年稍后发布的 AMD Athlon X2 架构。多核系统导致许多软件组件被重新设计，并影响了我们编写代码的方式。如今，几乎所有面向消费者的设备中的处理器都是多核 CPU。在撰写本书时，高端笔记本电脑包含超过十个物理核心，而服务器处理器包含的核心几乎达到 100 个。

这听起来可能非常令人印象深刻，但我们不能无限地增加核心。首先，每个核心在工作时会产生热量，并且安全地通过处理器封装从核心中散热仍然是一个挑战。这意味着当更多核心运行时，热量可能会迅速超过冷却能力。在这种情况下，多核处理器将降低时钟速度。可以看到具有大量核心的服务器芯片频率明显低于进入笔记本电脑和台式机的处理器的原因之一。

多核系统中的核心彼此连接，也连接到共享资源，例如末级缓存 (Last Level Cache) 和内存控制器 (Memory Controller)。这样的通信通道称为互连 (interconnect)，通常具有环形或网状拓扑结构。CPU 设计者面临的另一个挑战是随着核心数量的增加，保持机器的平衡。当复制核心时，一些资源保持共享，例如内存总线和末级缓存。这导致随着核心的增加，性能回报递减，除非您还解决了其他共享资源的吞吐量，例如互连带宽、末级缓存大小和带宽，以及内存带宽。共享资源经常成为多核系统中性能问题的来源。

3.5.2 同时多线程

改进多线程性能的一种更复杂的方法是同时多线程 (Simultaneous Multi-Threading, SMT)。人们经常使用术语超线程 (Hyperthreading) 来描述相同的事物。这种技术的目标是充分利用 CPU 流水线的可用宽度。SMT 允许多个软件线程在同一物理核心上使用共享资源同时运行。更准确地说，来自多个软件线程的指令在同一周期内同时执行。这些线程不必来自同一个进程；它们可以是完全不同的程序，恰好被调度在同一个物理核心上。

图 13 展示了在非 SMT 和 SMT2 处理器上的执行示例。在两种情况下，处理器流水线的宽度为四，每个插槽表示发出新指令的机会。100% 的机器利用率是指没有未使用的插槽，这在实际工作负载中从不会发生。很容易看出，在非 SMT 情况下，存在许多未使用的插槽，因此可用资源没有得到充分利用。这可能是由于多种原因引起的；一个常见的原因是缓存未命中。在周期 3 时，线程 1 由于等待数据到达而无法取得进展。SMT 处理器利用这个机会从另一个线程调度有用的工作。这里的目标是通过另一个线程占用未使用的插槽来隐藏内存延迟，提高硬件利用率和多线程性能。

在 SMT2 实现中，每个物理核心用两个逻辑核心表示，这些逻辑核心对操作系统显示为两个独立的处理器，可用于接受工作。考虑这样一种情况，我们有 16 个准备运行的软件线程，但只有 8 个物理核心。在非 SMT 系统中，只有 8 个线程将同时运行，而在 SMT2 中，我们可以同时执行所有 16 个线程。在另一种假设情况下，如果两个程序运行在一个启用了 SMT 的核心上，并且每个程序一直只利用了四个可用插槽中的两个，那么它们以与在该物理核心上独自运行时一样的速度运行的几率很高。

尽管两个程序运行在同一个处理器核心上，它们彼此完全分离。在启用了 SMT 的处理器中，即使指令混合在一起，它们也具有不同的上下文，有助于保持执行的正确性。为了支持 SMT，CPU 必须复制体系结构状态（程序计数器、寄存器）以保持线程上下文。其他 CPU 资源可以共享。在典型的实现中，缓存资源在硬件线程之间动态共享。用于跟踪乱序执行和推测执行的资源可以复制或分区。

在 SMT2 核心中，两个逻辑核心确实同时运行。在 CPU 前端，它们以交替的顺序（每个周期或几个周期）获取指令。在后端，处理器每个周期从所有线程中选择要执行的指令。指令执行混合，因为处理器动态地将执行单元在两个线

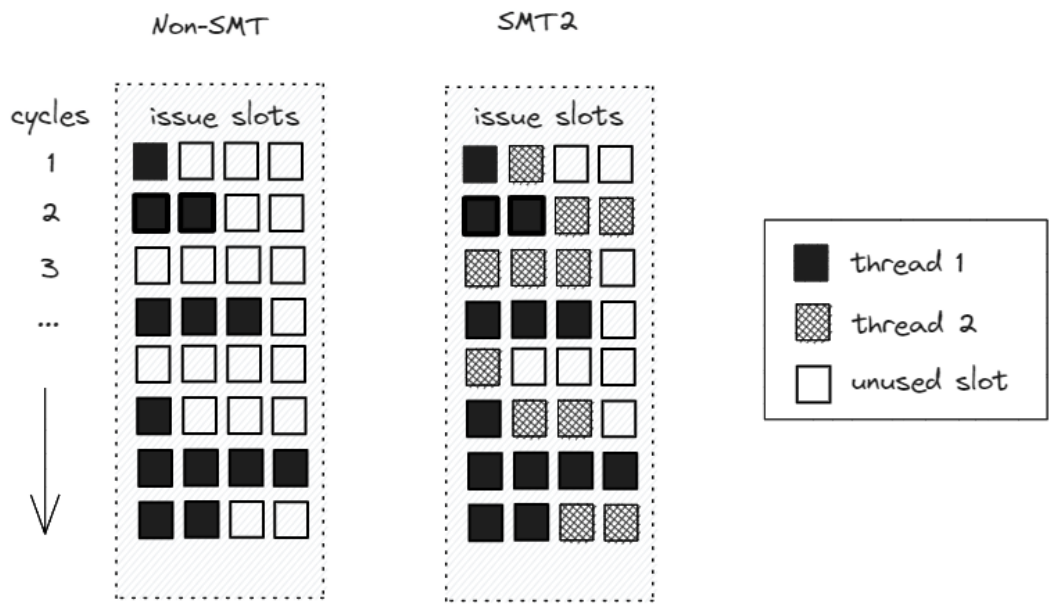


Figure 13: 在 4 宽度非 SMT 和 4 宽度 SMT2 处理器上的执行。

程之间调度。

因此，SMT 是一种非常灵活的设置，可以恢复未使用的 CPU 发射插槽。SMT 提供了相等的单线程性能，除了对多线程的好处外。现代多线程 CPU 支持两路（SMT2）或四路（SMT4）。

SMT 也有自己的缺点。由于某些资源在逻辑核心之间共享，它们最终可能会竞争使用这些资源。最有可能的 SMT 惩罚是由于对 L1 和 L2 缓存的竞争。由于它们在两个逻辑核心之间共享，它们可能在缓存中缺少空间，并迫使将将来将由另一个线程使用的数据逐出。

SMT 也给软件开发人员带来了很大的负担，因为它使得更难以预测和衡量在 SMT 核心上运行的应用程序的性能。想象一下，您在 SMT 核心上运行性能关键代码，突然操作系统在同一处理器的兄弟逻辑核心上放置了另一个要求严格的作业。您的代码几乎耗尽了机器的资源，现在您需要与其他人共享。在云环境中，这个问题特别突出，因为您无法预测您的应用程序是否会有嘈杂的邻居。

某些同时多线程实现存在安全问题。研究人员表明，一些早期实现存在漏洞，通过这些漏洞可以使一个应用程序从同一处理器的兄弟逻辑核心中监视其缓存使用，来窃取另一个应用程序的关键信息（如加密密钥）。我们不会深入探讨这个话题，因为硬件安全不在本书的范围内。

3.5.3 混合架构

计算机设计师们还开发了混合 CPU 设计，其中两种（或更多）类型的核心放置在同一处理器中。通常，更强大的核心与相对较慢的核心配对，以解决不同的目标。在这样的系统中，大核心用于延迟敏感的任务，而小核心则提供了较低的功耗。但是，两种类型的核心也可以同时使用，以提高多线程性能。所有核心都可以访问相同的内存，因此工作负载可以在大核心和小核心之间动态迁移。其目的是创建一个能够更好地适应动态计算需求并使用更少功耗的多核处理器。例如，视频游戏既有单核心突发性能的部分，也有可以扩展到多个核心的部分。

第一个主流的混合架构是 ARM 的 big.LITTLE，于 2011 年 10 月推出。其他供应商也采用了这种方法。苹果于 2020 年推出了其 M1 芯片，具有四个高性能的“Firestorm”核心和四个节能的“Icestorm”核心。英特尔于 2021 年推出了其 Alderlake 混合架构，顶级配置中配备了八个 P 核心和八个 E 核心。

混合架构结合了两种核心类型的优势，但它也带来了一系列挑战。首先，它要求核心完全兼容 ISA，即它们应该能

够执行相同的指令集。否则，调度就会受到限制。例如，如果大核心具有一些小核心上不可用的高级指令，那么您只能将大核心分配给使用这些指令的工作负载。这就是为什么通常供应商在选择混合处理器的 ISA 时使用“最大公约数”的方法。

即使具有 ISA 兼容的核心，调度也变得具有挑战性。不同类型的工作负载需要特定的调度方案，例如，突发执行与稳定执行，低 IPC 与高 IPC，低重要性与高重要性等。这很快就变得不那么简单了。以下是一些优化调度的考虑因素：

- 利用小核心以节省功耗。不要为后台工作唤醒大核心。
- 识别适合转移到较小核心的候选项（低重要性，低 IPC）。类似地，将高重要性，高 IPC 任务提升到大核心。
- 在分配新任务时，首先使用空闲的大核心。在 SMT 的情况下，使用两个逻辑线程都空闲的大核心。之后，使用空闲的小核心。之后，使用大核心的兄弟逻辑线程。

从程序员的角度来看，不需要对代码进行任何更改就可以利用混合系统。这种方法在面向客户的设备中变得非常流行，特别是在智能手机中。

3.6 存储器层次结构

为了有效地利用 CPU 中提供的所有硬件资源，需要在正确的时间提供正确的数据。理解存储器层次结构对于充分发挥 CPU 性能至关重要。大多数程序都表现出局部性的属性：它们不会均匀地访问所有代码或数据。CPU 存储器层次结构建立在两个基本属性上：

- **时间局部性**：当访问给定的内存位置时，很可能在不久的将来再次访问同一位置。理想情况下，我们希望在下一次需要时，该信息位于缓存中。
- **空间局部性**：当访问给定的内存位置时，很可能会在不久的将来访问附近的位置。这指的是将相关数据放在彼此附近。当程序从内存中读取一个字节时，通常会获取更大的内存块（缓存行 cache line），因为该程序很可能很快就会需要该数据。

本节概述了现代 CPU 支持的存储器层次结构系统的关键属性。

3.6.1 缓存层次结构

缓存是从 CPU 流水线发出的任何请求（用于代码或数据）的存储器层次结构的第一级。理想情况下，具有最小访问延迟的无限缓存是流水线的最佳选择。然而，在现实中，任何缓存的访问时间都会随着大小的增加而增加。因此，缓存被组织为一系列靠近执行单元的小型、快速的存储块，由较大、较慢的块支持。缓存层次结构的特定级别可以专门用于代码（指令缓存，i-cache）或数据（数据缓存，d-cache），或者在代码和数据之间共享（统一缓存）。此外，层次结构的某些级别可以是特定核心专用的，而其他级别可以在核心之间共享。

缓存被组织为具有定义的块大小（**缓存行 (cache line)**）。现代 CPU 中典型的缓存行大小为 64 字节。靠近执行流水线的缓存通常的大小范围从 8KiB 到 32KiB 不等。在层次结构中更远的缓存可以在 64KiB 到 16MiB 之间。任何级别的缓存的体系结构由以下四个属性定义。

3.6.1.1 数据在缓存中的放置

用于请求的地址用于访问缓存。在直接映射缓存中，给定块地址只能出现在缓存中的一个位置，并由下面的映射函数定义。

$$\text{缓存中的块数量} = \frac{\text{缓存大小}}{\text{缓存块大小}}$$

$$\text{直接映射位置} = (\text{块地址}) \bmod (\text{缓存中的块数量})$$

在完全关联的缓存中，给定块可以放置在缓存中的任何位置。

直接映射缓存和完全关联映射之间的中间选项是集合关联映射 (set-associative mapping)。在这样的缓存中，块被组织为集合，通常每个集合包含 2、4、8 或 16 个块。首先将给定地址映射到一个集合。在集合中，地址可以放置在该集合中的任何位置，即在该集合的块中。具有 m 个块的每个集合的缓存被描述为 m 路集合关联缓存 (m-way set-associative cache)。集合关联缓存的公式为：

$$\text{缓存中的集合数量} = \frac{\text{缓存中的块数量}}{\text{每组的块数量（关联性）}}$$

$$\text{集合 (m 路) 关联位置} = (\text{块地址}) \bmod (\text{缓存中的集合数量})$$

3.6.1.2 在缓存中查找数据 集合关联缓存中的每个块都与一个地址标签相关联。此外，标签还包含状态位，例如有效位，用于指示数据是否有效。标签还可以包含其他位，用于指示访问信息、共享信息等，这将在本章的后续部分描述。

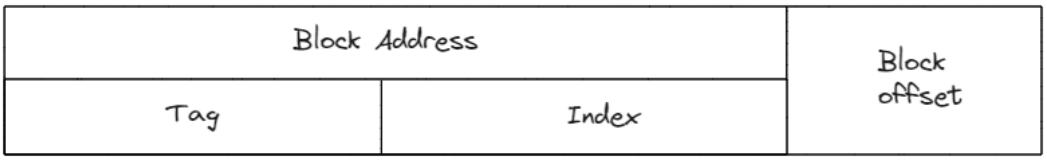


Figure 14: 用于缓存查找的地址组织。

图 14 显示了从流水线生成的地址如何用于检查缓存。最低位地址位定义了给定块内的偏移量；块偏移位（对于 32 字节缓存行为 5 位，对于 64 字节缓存行为 6 位）。使用上面描述的公式根据索引位选择集合。选择集合后，标签位用于与该集合中的所有标签进行比较。如果其中一个标签与传入请求的标签匹配并且有效位已设置，则会发生缓存命中。与该块条目相关联的数据（与标签查找并行读出缓存的数据数组）将提供给执行流水线。如果标签不匹配，则会发生缓存未命中。

3.6.1.3 处理未命中 当发生缓存未命中时，控制器必须选择要替换的缓存中的块，以分配导致未命中的地址。对于直接映射缓存，由于新地址只能分配到单个位置，因此取消分配映射到该位置的先前条目，并在其位置安装新条目。在集合关联缓存中，由于新的缓存块可以放置在集合的任何块中，因此需要替换算法。常用的替换算法是最近最少使用（LRU）策略，其中最近最少访问的块被淘汰以腾出空间以存放未命中地址。另一种选择是随机选择一个块作为受害块 (victim block)。大多数 CPU 在硬件中定义了这些功能，使执行软件更加容易。

3.6.1.4 处理写操作 相对于数据读取，对缓存的写入访问较少。在缓存中处理写入更加困难，CPU 实现使用各种技术来处理这种复杂性。软件开发人员应特别注意硬件支持的各种写缓存流程，以确保其代码的最佳性能。

CPU 设计使用两种基本机制来处理命中缓存的写操作：

- 在写透 (write-through) 缓存中，命中的数据被写入缓存中的块和层次结构中的下一级。
- 在写回 (write-back) 缓存中，命中的数据仅被写入缓存中。随后，层次结构的较低级别包含陈旧的数据。修改行的状态通过标签中的脏位进行跟踪。当修改的缓存行最终从缓存中淘汰时，写回操作会强制将数据写回到下一级。

写操作导致的缓存未命中可以通过两种方式处理：

- 在写分配或写未命中获取 (write-allocate or fetch on write miss) 缓存中，从层次结构的下一级加载未命中位置的数据到缓存中，随后处理写操作，就像写命中一样。
- 如果缓存使用非写分配策略 (no-write-allocate policy)，则将缓存未命中事务直接发送到层次结构的下一级，并且该块不加载到缓存中。

在这些选项中，大多数设计通常选择实现具有写分配策略的写回缓存，因为这两种技术都尝试将后续写事务转换为缓存命中，而无需向下一级发送额外的流量。写透写缓存通常使用非写分配策略。

3.6.1.5 **其他缓存优化技术** 对于程序员来说，理解缓存层次结构的行为对于从任何应用程序中提取性能至关重要。从流水线的角度来看，访问任何请求的延迟由以下公式给出，该公式可以递归应用到缓存层次结构的所有级别，直到主存：

$$\text{平均访问延迟} = \text{命中时间} + \text{未命中率} \times \text{未命中惩罚}$$

硬件设计者面临的挑战是通过许多新颖的微体系结构技术来减少命中时间和未命中惩罚。从根本上说，缓存未命中会阻塞流水线并损害性能。任何缓存的未命中率高度依赖于缓存体系结构（块大小、关联性）和机器上运行的软件。因此，优化未命中率成为硬件-软件协同设计的工作。正如前面讨论的，CPU 为缓存提供了最佳的硬件组织。下面描述了可以在硬件和软件中实现的降低缓存未命中率的其他技术。

3.6.1.5.1 **硬件和软件预取** 为了避免缓存未命中及随后的停顿，一种方法是在流水线需求 (demand) 之前将指令和数据预取到不同级别的缓存层中。假设如果预取请求在流水线中足够提前发出，那么处理未命中惩罚的时间就可以被隐藏。大多数 CPU 提供了隐式的硬件预取功能，程序员可以通过显式的软件预取来控制。

硬件预取器观察正在运行的应用程序的行为，并在缓存未命中的重复模式上启动预取。硬件预取可以自动适应应用程序的动态行为，例如不同的数据集，并且不需要优化编译器或分析支持。此外，硬件预取工作而不需要额外的地址生成和预取指令的开销。然而，硬件预取仅限于学习和预取有限集的缓存未命中模式。

软件内存预取补充了硬件进行的预取。开发人员可以通过专用的硬件指令（见 Section 8.2）指定提前需要的内存位置。编译器还可以自动将预取指令添加到代码中，以请求使用之前的数据。预取技术需要在需求 (demand) 和预取请求之间进行平衡，以防止预取流量减慢需求流量。

3.6.2 主存储器

主存储器是缓存之后的下一个层次，由内存控制器单元（MCU）发起对数据的加载和存储请求。过去，这个电路位于主板上的北桥芯片中。但是现在，大多数处理器都将此组件嵌入其中，因此 CPU 与主存储器之间有一个专用的内存总线连接。

主存储器使用 DRAM（动态随机存取存储器）技术，支持以合理的成本点获取大容量。在比较 DRAM 模块时，人们通常会关注内存密度和内存速度，当然还有其价格。内存密度定义了模块拥有的内存量，以 GB 为单位。显然，可用内存越多越好，因为它是操作系统和应用程序使用的宝贵资源。

主存储器的性能由延迟和带宽描述。内存延迟是发出内存访问请求和 CPU 可用于使用数据之间经过的时间。内存带宽定义了在一定时间内可以获取多少字节，通常以每秒字节数（GB/s）表示。

3.6.2.1 **DDR** DDR（双倍数据速率）DRAM 技术是大多数 CPU 支持的主要 DRAM 技术。从历史上看，DRAM 的带宽每一代都在提高，而 DRAM 的延迟却保持不变甚至增加。表 2 显示了过去三代 DDR 技术的最高数据速率、峰值带宽以及相应的读取延迟。数据速率以每秒传输的百万次（MT/s）为单位。此表中显示的延迟对应于 DRAM 器件本身的延迟。通常，由于缓存控制器、内存控制器和芯片内连接引起的额外延迟和排队延迟，从 CPU 流水线（在加载到使用的缓存未命中时）看到的延迟较高（在 50 纳秒至 150 纳秒范围内）。您可以在 Section 4.10中看到测量观察到的内存延迟和带宽的示例。

Table 2: 过去三代 DDR 技术的性能特征。

DDR	年份	最高数据速率 (MT/s)	峰值带宽 (GB/s)	设备内读取延迟 (ns)
DDR3	2007	2133	17.1	10.3
DDR4	2014	3200	25.6	12.5
DDR5	2020	6400	51.2	14

值得一提的是，DRAM 芯片需要定期刷新其内存单元。这是因为位值被存储为微小电容器上的电荷存在，所以它可能会随着时间的推移失去电荷。为了防止这种情况发生，有特殊的电路读取每个单元并将其写回，有效地恢复电容器的电荷。当 DRAM 芯片处于刷新过程中时，它不会响应内存访问请求。

DRAM 模块组织为一组 DRAM 芯片。内存 rank 是一个术语，用于描述模块上存在多少组 DRAM 芯片。例如，单 rank（1R）内存模块包含一组 DRAM 芯片。双 rank（2R）内存模块有两组 DRAM 芯片，因此将单 rank 模块的容量加倍。同样，还可以购买四 rank（4R）和八 rank（8R）内存模块。

每个 rank 由多个 DRAM 芯片组成。内存 width 定义了每个 DRAM 芯片的总线宽度。由于每个 rank 的总线宽度为 64 位（或 ECC RAM 为 72 位），它还定义了 rank 内存在芯片中的数量。内存宽度可以是`x4`、`x8`或`x16`中的一个值，这定义了发送到每个芯片的总线宽度。例如，图 15 显示了一个 2Rx16 双 rank DRAM DDR4 模块的组织，总容量为 2GB。每个 rank 中有四个芯片，总线宽度为 16 位。四个芯片共同提供 64 位输出。两个 rank 通过一个 rank 选择信号逐个选择。

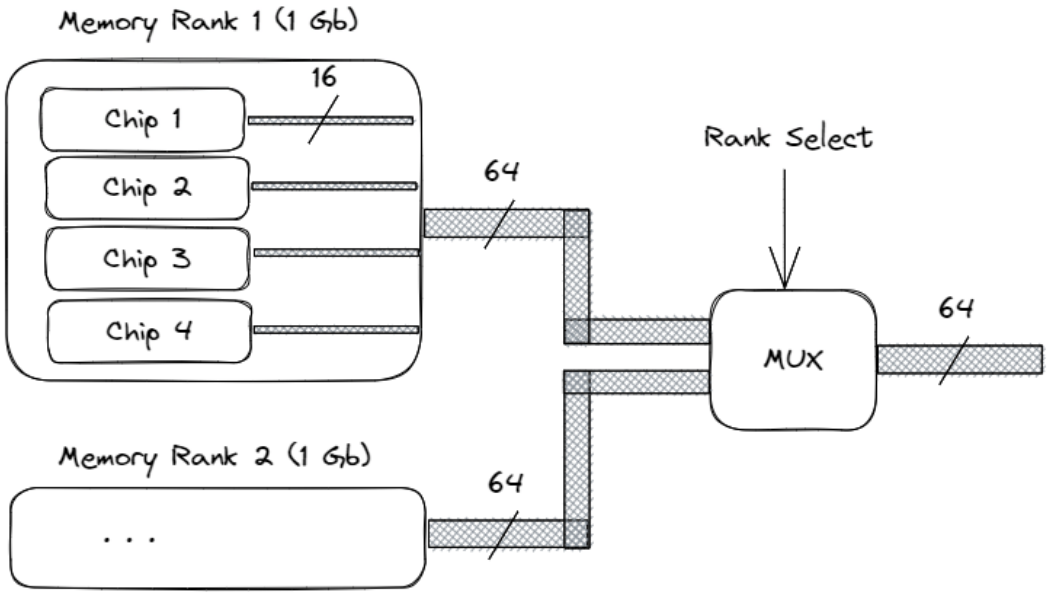


Figure 15: 2Rx16 双 rank DRAM DDR4 模块，总容量 2GB 的组织。

单 rank 或双 rank 的性能哪个更好并没有直接的答案，因为它取决于应用程序的类型。通过 rank 选择信号从一个 rank 切换到另一个 rank 需要额外的时钟周期，这可能会增加访问延迟。另一方面，如果一个 rank 没有被访问，它可以在其他 rank 忙碌时并行进行刷新周期。一旦上一个 rank 完成数据传输，下一个 rank 就可以立即开始传输。此外，单 rank 模块产生的热量更少，故障的可能性更低。

进一步说，我们可以在系统中安装多个 DRAM 模块，不仅增加内存容量，还增加内存带宽。多个内存通道的设置用于扩展内存控制器和 DRAM 之间的通信速度。

一个只有一个内存通道的系统，在 DRAM 和内存控制器之间有一个 64 位宽的数据总线。多通道架构增加了内存总线的宽度，允许同时访问 DRAM 模块。例如，双通道架构将内存数据总线的宽度从 64 位扩展到 128 位，将可用带宽加倍，见图 16。请注意，每个内存模块仍然是一个 64 位设备，但连接它们的方式不同。如今，服务器通常具有四个和八个内存通道。

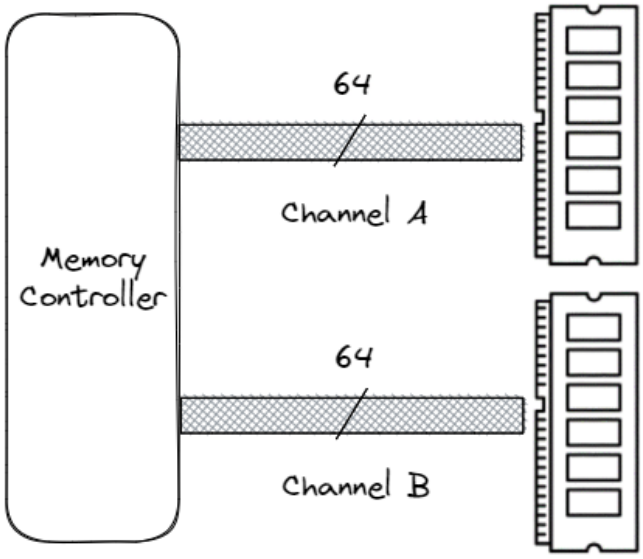


Figure 16: 双通道 DRAM 设置的组织。

或者，您也可能遇到具有复制内存控制器的设置。例如，处理器可能具有两个集成的内存控制器，每个内存控制器都可以支持多个内存通道。这两个控制器是独立的，只查看总物理内存地址空间的自己的部分。

我们可以通过以下简单的公式快速计算给定内存技术的最大内存带宽：

$$\text{最大内存带宽} = \text{数据速率} \times \text{每周期字节数}$$

例如，对于单通道 DDR4 配置，数据速率为`2400 MT/s`，每个内存周期可以传输 64 位（8 字节），因此最大带宽等于`2400 * 8 = 19.2 GB/s`。双通道或双内存控制器设置将带宽加倍至`38.4 GB/s`。但要记住，这些数字是假设数据传输将在每个内存时钟周期中发生的理论最大值，在实践中实际上是不会发生的。因此，当测量实际内存速度时，您将始终看到比最大理论传输带宽低的值。

要启用多通道配置，您需要具备支持这种架构的 CPU 和主板，并在主板上正确的内存插槽中安装相同数量的内存模块。在 Windows 上检查设置的最快方法是运行诸如`CPU-Z`或`HwInfo`之类的硬件识别实用程序；在 Linux 上，您可以使用`dmidecode`命令。或者，您可以运行内存带宽基准测试，例如 Intel 的`mlc`或`Stream`。

要在系统中利用多个内存通道，有一种称为交错 (interleaving) 的技术。它在一个页面中将相邻地址在多个内存设备之间分布。图 17 显示了用于顺序内存访问的 2 路交错的示例。与以前一样，我们有双通道内存配置（通道 A 和 B），具有两个独立的内存控制器。现代处理器按每四个缓存行（256 字节）进行交错，即，前四个相邻缓存行发送到通道 A，然后下一组四个缓存行发送到通道 B。

如果不使用交错，连续的相邻访问将发送到同一个内存控制器，而不是利用第二个可用的控制器。相反，交错使硬件并行性更好地利用了可用的内存带宽。对于大多数工作负载，当所有通道都被填充时，性能最大化，因为它将单个内存区域扩展到尽可能多的 DRAM 模块中。

虽然增加内存带宽通常是有益的，但它并不总是转化为更好的系统性能，而且高度依赖于应用程序。另一方面，注意可用和已使用的内存带宽非常重要，因为一旦它成为主要瓶颈，应用程序就会停止扩展，即，添加更多核心并不

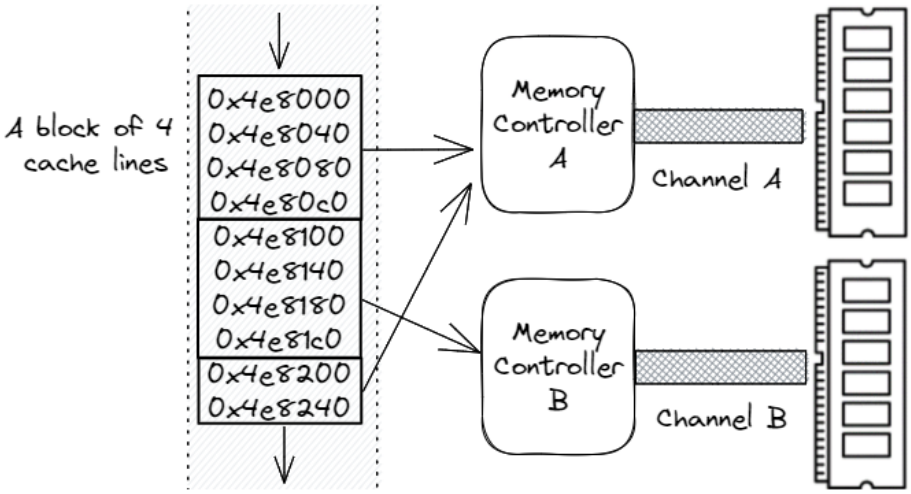


Figure 17: 顺序内存访问的 2 路交错。

能使其运行更快。

3.6.2.2 **GDDR 和 HBM** 除了多通道 DDR 外，还有其他技术针对需要更高内存带宽以实现更高性能的工作负载。GDDR（图形 DDR）和 HBM（高带宽内存）等技术是最显着的技术。它们在高端图形、高性能计算（如气候建模、分子动力学、物理模拟），但也包括自动驾驶和人工智能/机器学习等领域中得到应用。它们在这些领域中非常适用，因为这些应用需要非常快速地移动大量数据。

GDDR 最初是为图形设计的，现在几乎每个高性能图形卡都在使用。虽然 GDDR 与 DDR 共享一些特征，但它也有很大的不同。虽然 DRAM DDR 设计用于更低的延迟，但 GDDR 则设计用于更高的带宽，因为它位于处理器芯片本身的同一封装中。与 DDR 类似，GDDR 接口每个时钟周期传输两个 32 位字（总共 64 位）。最新的 GDDR6X 标准可以实现高达 168 GB/s 的带宽，以相对较低的 656 MHz 频率运行。

HBM 是一种新型的 CPU/GPU 内存，它垂直堆叠内存芯片，也称为 3D 堆叠。与 GDDR 类似，HBM 极大地缩短了数据到达处理器的距离。与 DDR 和 GDDR 的主要区别在于，HBM 内存总线非常宽：每个 HBM 堆栈为 1024 位。这使 HBM 能够实现超高带宽。最新的 HBM3 标准支持每个封装高达 665 GB/s 的带宽。它还以 500 MHz 的低频率运行，并具有每个封装高达 48 GB 的内存密度。

如果您想获得尽可能多的内存带宽，那么拥有 HBM 的系统将是一个不错的选择。但是，在撰写本文时，这项技术相当昂贵。由于 GDDR 主要用于图形卡，HBM 可能是加速在 CPU 上运行的某些工作负载的好选择。事实上，第一批具有集成 HBM 的 x86 通用服务器芯片现已上市。

3.7 虚拟内存

虚拟内存是一种机制，将连接到 CPU 的物理内存与在 CPU 上执行的所有进程共享。虚拟内存提供了一种保护机制，可以防止其他进程访问分配给特定进程的内存。虚拟内存还提供重定位，即能够在物理内存中的任何位置加载程序而不更改程序中的地址。

在支持虚拟内存的 CPU 中，程序使用虚拟地址进行访问。但是，虽然用户代码在虚拟地址上运行，但从内存中检索数据需要物理地址。此外，为了有效管理稀缺的物理内存，它被划分为页面。因此，应用程序在一组由操作系统提供的页面上运行。

访问数据和代码（指令）都需要地址转换。具有 4KB 页面大小的系统的机制如图 18 所示。虚拟地址分为两部分。虚拟页号（52 个最高有效位）用于索引页表，以生成虚拟页号和相应物理页之间的映射。对于 4KB 页面中的偏移量，

我们需要 12 位；正如已经说过的那样，64 位指针的其他 52 位用于页本身的地址。请注意，页面内的偏移量（12 个最低有效位）不需要转换，并且“原样”用于访问物理内存位置。

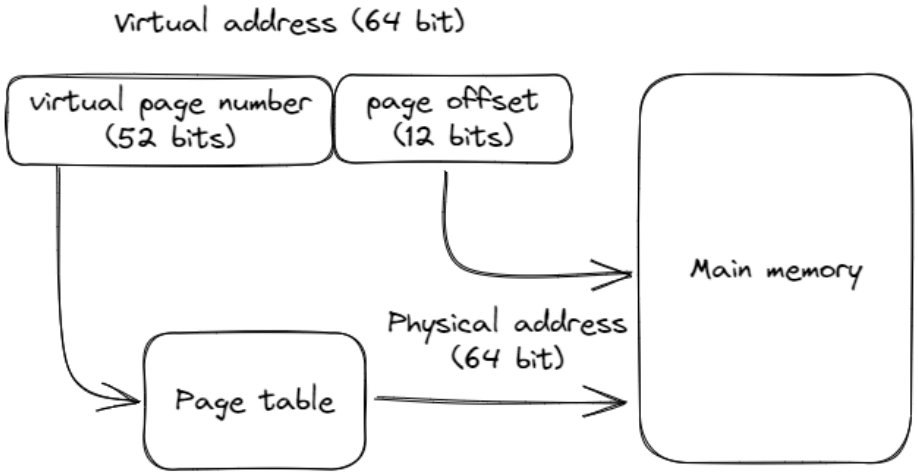


Figure 18: 4KB 页面的虚拟到物理地址转换

页表可以是单层或嵌套的。图 19 显示了一个 2 级页表的示例。请注意地址如何分成更多部分。首先要提的是，没有使用 16 个最高有效位。这似乎浪费了位，但即使使用剩余的 48 位，我们也可以寻址 256 TB 的总内存 (2^{48})。一些应用程序使用这些未使用的位来保留元数据，也称为“指针标记”。

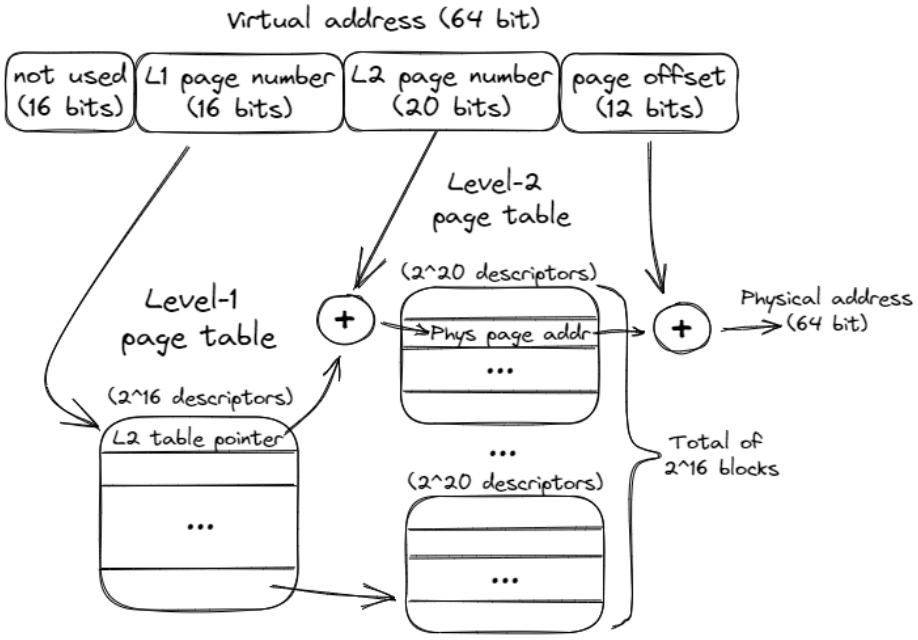


Figure 19: 2 级页表示例

嵌套页表是一个 radix 树，它与一些元数据一起保存物理页地址。要找到这样一个 2 级页表的翻译，我们首先使用位 32..47 作为索引到 1 级页表，也称为“页表目录”。目录中的每个描述符都指向 2^{16} 个 2 级表块之一。一旦找到合适的 L2 块，我们就使用位 12..31 来找到物理页地址。将其与页偏移量 (位 0..11) 连接起来，我们就得到了物理地址，可以用来从 DRAM 检索数据。

页表的确切格式由 CPU 决定，原因将在接下来的几个段落中讨论。因此，页表组织的变化仅限于 CPU 支持的内容。

如今，通常可以看到 4 级和 5 级页表。现代 CPU 支持具有 48 位指针 (256 TB 总内存) 的 4 级页表和具有 57 位指针 (128 PB 总内存) 的 5 级页表。

将页表分为多个级别不会改变可寻址内存的总量。但是，嵌套方法不需要将整个页表存储为连续数组，也不分配没有描述符的块。这节省了内存空间，但增加了遍历页表的开销。

无法提供物理地址映射称为“页面错误”。如果请求的页面无效或当前不在主内存中，就会发生这种情况。两个最常见的原因是：1) 操作系统承诺分配一个页面，但尚未用物理页面支持它，以及 2) 访问的页面被换出到磁盘并且当前没有存储在 RAM 中。

3.7.1 转译后备缓冲区 (TLB)

在分层页表中搜索可能代价高昂，需要遍历层次结构，可能进行多次间接访问。这种遍历通常称为页行走 (page walk)。为了减少地址翻译时间，CPU 支持一种称为转译后备缓冲区 (translation lookaside buffer, TLB) 的硬件结构来缓存最近使用的翻译。类似于普通缓存，TLB 通常设计为 L1 ITLB（指令）、L1 DTLB（数据）的层次结构，然后是共享的 (指令和数据) L2 STLB。为了降低内存访问延迟，TLB 和缓存查找并行发生，因为数据缓存使用虚拟地址操作，不需要预先进行地址翻译。

TLB 层次结构为相对较大的内存空间保留翻译。但是，TLB 未命中可能会非常昂贵。为了加快对 TLB 未命中处理，CPU 具有一个称为硬件页行走器 (HW page walker) 的机制。这样的单元可以通过发出所需的指令来遍历页表，直接在硬件中执行页行走，而不会中断内核。这就是页表格式由 CPU 决定，操作系统必须遵守的原因。高端处理器有多个硬件页行走器，可以同时处理多个 TLB 未命中。然而，即使使用了现代 CPU 提供的所有加速，TLB 未命中仍然会为许多应用程序造成性能瓶颈。

3.7.2 大页

使用较小的页面大小可以更有效地管理可用内存并减少碎片化。然而，缺点是它需要更多的页表条目来覆盖相同的内存区域。考虑两种页面大小：4KB（x86 上的默认大小）和 2MB 的“大页”大小。对于处理 10MB 数据的应用程序，在第一种情况下需要 2560 个条目，而如果将地址空间映射到巨大页面，只需要 5 个条目。这些在 Linux 上称为“Huge Pages”，FreeBSD 上称为“Super Pages”，Windows 上称为“Large Pages”，但它们都表示同一个意思。在本书的其余部分，我们将它们称为 Huge Pages。

指向 Huge Page 中数据的地址示例如图 20 所示。与默认页面大小一样，使用 Huge Pages 时的确切地址格式由硬件决定，但幸运的是，我们作为程序员通常不必担心这一点。

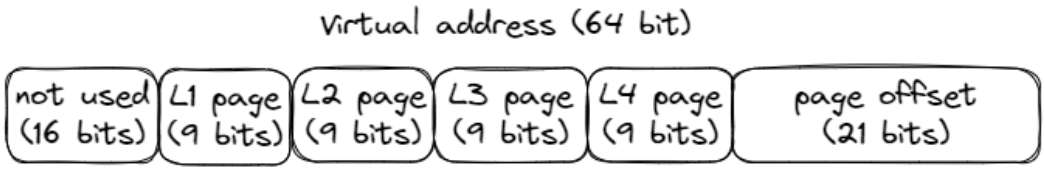


Figure 20: 指向 2MB 页面中数据的虚拟地址

使用 Huge Pages 可以大大减少对 TLB 层次结构的压力，因为需要的 TLB 条目更少。它大大增加了 TLB 命中率。我们将在 Section 8.4 和 Section 11.8 中讨论如何使用 Huge Pages 减少 TLB 未命中率。使用 Huge Pages 的缺点是内存碎片化，并且在某些情况下，由于操作系统更难管理大量内存块并确保有效利用可用内存，非确定性页面分配延迟会增加。要在运行时满足 2MB Huge Page 分配请求，操作系统需要找到 2MB 的连续块。如果找不到，操作系统需要重组页面，从而导致更长的分配延迟。

3.8 现代 CPU 设计

为了看到我们在本章讨论的所有概念如何在实践中使用，让我们来看看英特尔第 12 代酷睿处理器 Goldencove 的实现，该处理器于 2021 年上市。该核心被用作 Alderlake 和 Sapphire Rapids 平台中的 P 核心。图 21 显示了 Goldencove 核心的模块图。请注意，本节仅描述了单个核心，而不是整个处理器。因此，我们将跳过关于频率、核心数量、L3 缓存、核心互连、内存延迟和带宽以及其他内容的讨论。

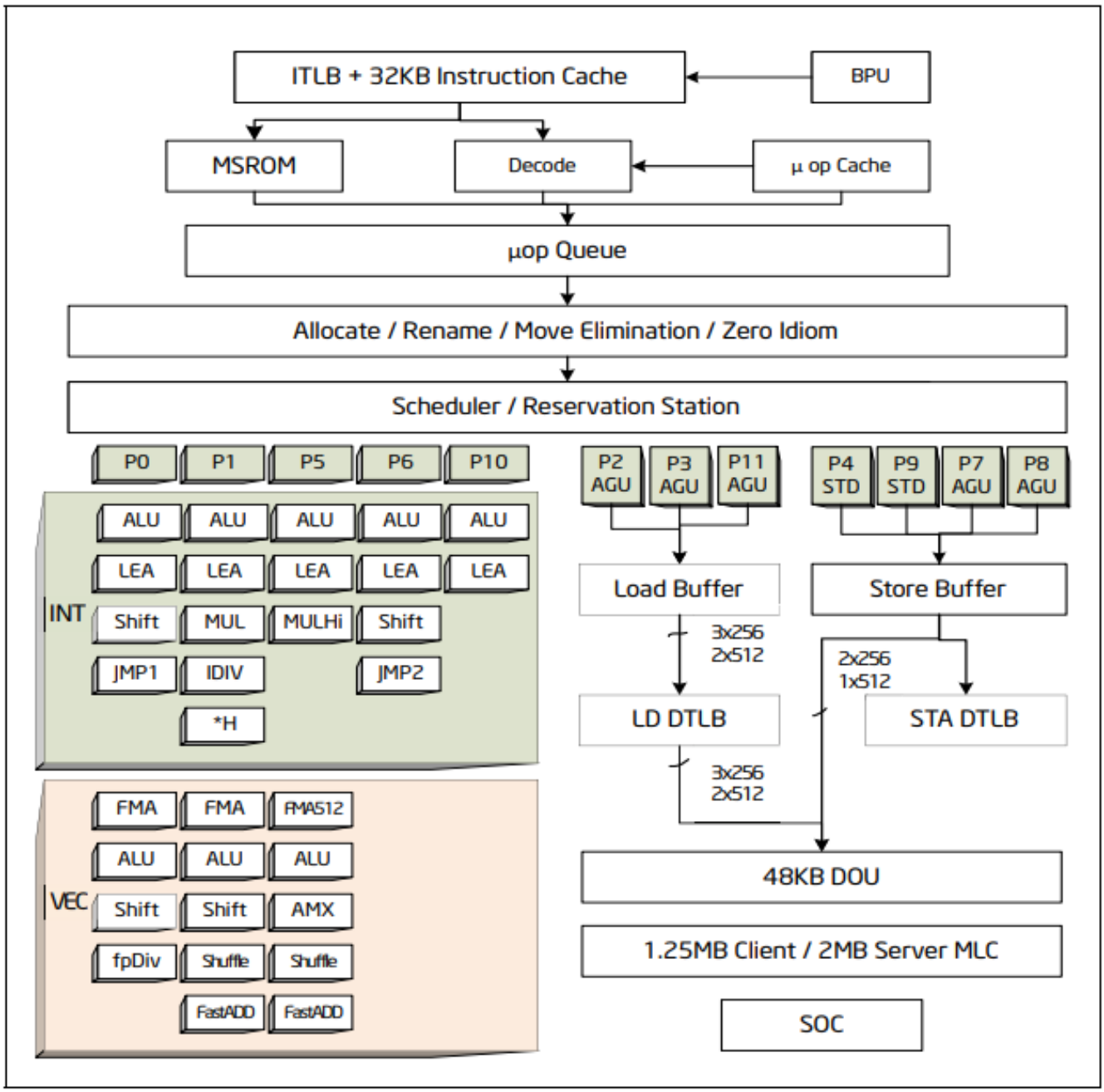


Figure 21: Intel GoldenCove 微架构 CPU 核心的模块图。© 图片来源 [Intel, 2023b]。

该核心分为一个按顺序执行的前端，负责从内存中提取和解码 x86 指令为 μops，以及一个 6 宽度的超标量、乱序执行的后端。Goldencove 核心支持 2 路 SMT。它有一个 32KB 的一级指令缓存（L1 I-cache），和一个 48KB 的一级数据缓存（L1 D-cache）。L1 缓存由统一的 1.25MB（服务器芯片中为 2MB）L2 缓存支持。L1 和 L2 缓存对每个核心是私有的。在本节末尾，我们还将查看 TLB 层次结构。

3.8.1 CPU 前端

CPU 前端由一些数据结构组成，用于从内存中提取和解码指令。其主要目的是向 CPU 后端提供准备好的指令，后者负责实际执行指令。

技术上讲，指令提取是执行指令的第一阶段。但一旦程序达到稳定状态，分支预测单元（BPU）就会引导 CPU 前端的工作。这就是从 BPU 到指令缓存的箭头的原因。BPU 预测所有分支指令的方向，并根据这个预测引导下一个指令提取。

BPU 的核心是一个包含 12K 条目的分支目标缓冲区（BTB），其中包含有关分支及其目标的信息。这些信息被预测算法使用。每个周期，BPU 生成下一个提取地址，并将其传递给 CPU 前端。

CPU 前端每个周期从 L1 I-cache 中提取 32 字节的 x86 指令。这在两个线程之间共享，因此每个线程每隔一个周期会获得 32 字节。这些是复杂的、可变长度的 x86 指令。首先，预解码阶段通过检查指令来确定和标记可变指令的边界。在 x86 中，指令长度可以从 1 字节到 15 字节不等。该阶段还识别分支指令。预解码阶段将多达 6 条指令（也称为宏指令）移动到指令队列（图表中未显示）中，该队列在两个线程之间划分。指令队列还支持一个宏指令融合单元，它检测到两个宏指令可以融合成一个单一的微操作（μop）。这种优化可以节省流水线中的带宽。

稍后，多达六个预解码指令每个周期从指令队列发送到解码器单元。两个 SMT 线程每个周期交替访问此接口。6 路解码器将复杂的宏操作转换为固定长度的 μops。解码后的 μops 被排队到指令解码队列（IDQ），在图表上标记为“μop 队列”。

前端的一个主要性能提升特性是解码流缓冲区（DSB）或 μop 缓存。其动机是在与 L1 I-cache 并行工作的单独结构中缓存宏操作到 μops 的转换。当 BPU 生成一个新地址进行提取时，也会检查 DSB，以查看 μops 的转换是否已经在 DSB 中可用。频繁发生的宏操作会命中 DSB，流水线将避免为 32 字节的捆绑重复执行昂贵的预解码和解码操作。DSB 每个周期可以提供八个 μops，并且最多可以容纳 4K 个条目。

一些非常复杂的指令可能需要比解码器处理的 μops 更多。这些指令的 μops 来自微码顺序器（MSROM）。这些指令的示例包括用于字符串操作、加密、同步等的 HW 操作支持。此外，MSROM 保留了处理异常情况的微码操作，例如分支预测失败（需要流水线刷新）、浮点辅助（例如，当指令与非规范化的浮点值进行操作时）等。MSROM 每个周期可以向 IDQ 推送最多 4 个 μops。

指令解码队列（IDQ）提供了顺序 CPU 前端和乱序 CPU 后端之间的接口。IDQ 按顺序排列 μops，并且每个逻辑处理器在单线程模式下可以容纳 144 个 μops，在 SMT 活跃时每个线程可以容纳 72 个 μops。这是顺序 CPU 前端结束并且乱序 CPU 后端开始的地方。

3.8.2 CPU 后端

CPU 后端采用乱序执行引擎执行指令并存储结果。CPU 后端的核心是 512 条目的重排序缓冲区（ROB）。该单元在图表中被称为“分配/重命名 (Allocate / Rename)”。它有几个作用。首先，它提供寄存器重命名。只有 16 个通用整数寄存器和 32 个向量/SIMD 体系结构寄存器，但是物理寄存器的数量要多得多[43]。物理寄存器位于称为物理寄存器文件（PRF）的结构中。从体系结构可见寄存器到物理寄存器的映射保存在寄存器别名表（RAT）中。

其次，ROB 分配执行资源。当一条指令进入 ROB 时，将分配一个新条目，并为其分配资源，主要是一个执行单元和目标物理寄存器。ROB 每个周期可以分配多达 6 个 μops。

第三，ROB 跟踪推测执行。当一条指令完成其执行时，其状态会更新，并且会保留在那里，直到前面的指令也完成。之所以这样做，是因为指令总是按程序顺序退役。一旦一条指令退役，其 ROB 条目将被释放，并且指令的结果变得可见。退役阶段比分配阶段更宽：ROB 每个周期可以退役 8 条指令。

[43] 大约有 300 个物理通用寄存器（GPRs）和类似数量的向量寄存器。实际寄存器数量未公开。

处理器以特定方式处理某些操作，通常称为习惯用法，这些操作不需要或成本较低。处理器识别这些情况并允许它们比常规指令运行得更快。以下是一些这种情况：

- **置零（Zeroing）**：为了将零赋值给一个寄存器，编译器通常使用`XOR / PXOR / XORPS / XORPD`等指令，例如`XOR RAX, RAX`，编译器更喜欢使用这些指令，而不是等效的`MOV RAX, 0x0`指令，因为 XOR 编码使用的编码字节较少。这种置零习惯用法不会像其他常规指令一样执行，而是在 CPU 前端解析，这样可以节省执行资源。指令随后像通常一样被退役 (retires)。
- **移动消除（Move elimination）**：类似于前一个，寄存器到寄存器的 mov 操作，例如`MOV RAX, RBX`，可以在零周期延迟内执行。
- **NOP 指令**：`NOP`通常用于填充或对齐的目的。它只是被标记为已完成，而不将其分配到保留站 (Reservation Station)。
- **其他旁路（Other bypasses）**：CPU 设计架构师还优化了某些算术操作。例如，任何数乘以一始终得到相同的数。除以一也是如此。任何数乘以零始终得到零，等等。某些 CPU 可以在运行时识别这些情况，并以比常规乘法或除法更短的延迟执行它们。

“调度器/保留站 (Scheduler / Reservation Station)”（RS）是跟踪给定 μop 的所有资源可用性并在准备就绪时将 μop 分派到分配端口的结构。当一条指令进入 RS 时，调度器开始跟踪其数据依赖关系。一旦所有源操作数可用，RS 尝试将 μop 分派到空闲的执行端口。RS 的条目比 ROB 少。它每个周期最多可以分派 6 个 μops。

如图 21 所示，有 12 个执行端口：

- 端口 0、1、5、6 和 10 提供所有整数（INT）以及浮点和向量（VEC/FP）操作。分派到这些端口的指令不需要内存操作。
- 端口 2、3 和 11 用于地址生成（AGU）和加载操作。
- 端口 4 和 9 用于存储操作（STD）。
- 端口 7 和 8 用于地址生成。

分派的算术操作可以进入 INT 或 VEC/FP 执行端口。整数和向量/FP 寄存器堆栈位于不同位置。从 INT 堆栈到 VEC/FP 以及反之的操作（例如，转换、提取或插入）会带来额外的惩罚。

3.8.3 Load-Store Unit

Goldencove 核心每个周期最多可以执行三次加载和两次存储操作。一旦加载或存储操作离开调度器，加载-存储（LS）单元负责访问数据并将其保存在寄存器中。LS 单元有一个加载队列（LDQ，标记为“加载缓冲区”）和一个存储队列（STQ，标记为“存储缓冲区”），它们的大小未公开[44] 。LDQ 和 STQ 都在调度器分派时接收操作。

当有内存加载请求时，LS 使用虚拟地址查询 L1 缓存，并在 TLB 中查找物理地址转换。这两个操作同时启动。L1 D-cache 的大小为 48KB。如果两个操作都命中，则加载将数据传递给整数单元或浮点单元，并离开 LDQ。类似地，存储将数据写入数据缓存并退出 STQ。

在发生 L1 未命中时，硬件会启动对（私有）L2 缓存标签的查询。L2 缓存有两种变体：客户端为 1.25MB，服务器处理器为 2MB。在查询 L2 缓存时，会分配一个 64 字节宽的填充缓冲区条目（FB），一旦缓存行到达，它将保留缓存行。Goldencove 核心有 16 个填充缓冲区。为了降低延迟，同时会向 L3 缓存发送一次猜测查询，与 L2 缓存查找并行进行。

如果两个加载访问相同的缓存行，它们将命中相同的 FB。这样的两个加载将被“粘合 (glued)”在一起，只会启动一个内存请求。LS 单元动态重新排序操作，支持旧加载操作绕过旧加载和旧非冲突存储操作绕过旧加载。此外，LS 单元在存在包含加载的所有字节的较旧存储并且存储的数据已生成并在存储队列中可用时，支持存储到加载的转发。

[44] LDQ 和 STQ 的大小未公开，但人们已经测量过分别为 192 和 114 条目。

如果确认 L2 未命中，则加载将继续等待 L3 缓存的结果，这会产生更高的延迟。从那时起，请求离开核心并进入“uncore”，这是您可能经常在性能分析工具中看到的术语。核心中未完成的未命中在超级队列（SQ，图中未显示）中被跟踪，该队列可以跟踪高达 48 个 uncore 请求。在 L3 未命中的情况下，处理器开始设置内存访问。进一步的细节超出了本章的范围。

当发生存储时，在一般情况下，要修改一个内存位置，处理器需要加载完整的缓存行，对其进行更改，然后将其写回内存。如果要写入的地址不在缓存中，则需要执行与加载类似的机制将数据带入。在将数据写入缓存层之前，存储不能完成。

当然，存储操作也有一些优化。首先，如果我们处理的是一个存储或多个相邻存储（也称为流存储 (streaming stores)），这些存储修改了整个缓存行，则无需首先读取数据，因为所有字节都将被覆盖。因此，处理器将尝试组合写入以填充整个缓存行。如果成功，根本不需要内存读取操作。

其次，写入组合使多个存储组装在一起，并作为一个单元写入缓存层次结构。因此，如果多个存储修改同一缓存行，则只会向内存子系统发出一个内存写入请求。现代处理器具有称为存储缓冲区 (store buffer) 的数据结构，该结构尝试合并存储。存储指令将数据从寄存器复制到存储缓冲区。从那里，它可以写入 L1 缓存，或者它可以与其他存储组合到同一缓存行。存储缓冲区的容量有限，因此它只能暂时保存对缓存行的部分写入的请求。然而，当数据在存储缓冲区等待写入时，其他加载指令可以直接从存储缓冲区读取数据（存储到加载的转发 (store-to-load forwarding)）。

最后，如果我们在覆盖数据之前读取数据，缓存行通常会保留在缓存中，替换其他行。通过使用非临时 (non-temporal) 存储，可以改变这种行为，这是一种特殊的 CPU 指令，不会保留修改后的行在缓存中。在我们知道一旦更改数据就不再需要数据的情况下，非临时存储有助于更有效地利用缓存空间。非临时存储通过不驱逐其他可能很快需要的数据来帮助更有效地利用缓存空间。

3.8.4 TLB 层次结构

回想一下 Section 3.7.1，虚拟地址到物理地址的转换被缓存在 TLB 中。Golden Cove 的 TLB 层次结构如图 22 所示。与常规数据缓存类似，它有两个级别，其中级别 1 分别为指令（ITLB）和数据（DTLB）有单独的实例。L1 ITLB 有 256 个条目，用于常规的 4KB 页面，覆盖 256 * 4KB 等于 1MB 的内存空间，而 L1 DTLB 有 96 个条目，覆盖 384KB。

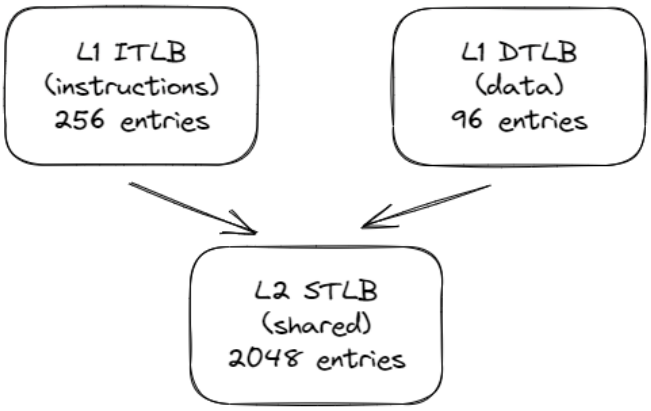


Figure 22: Golden Cove 的 TLB 层次.

第二级别的层次结构（STLB）缓存了指令和数据的转换。这是一个更大的存储，用于为在 L1 TLB 中未命中的请求提供服务。L2 STLB 可以容纳 2048 个最近的数据和指令页面地址转换，覆盖总共 8MB 的内存空间。对于 2MB 的大页面，可用的条目较少：L1 ITLB 有 32 个条目，L1 DTLB 有 32 个条目，而 L2 STLB 只能使用 1024 个条目，这些条目也是共享的常规 4KB 页面。

如果在 TLB 层次结构中找不到转换，则必须通过“行走 (walking)”内核页表来检索。有一种机制可以加速这种情况，

称为硬件页行走器。回想一下，页表是一个根据子表构建的基数树，其中子表的每个条目都包含指向树下一级的指针。

加速页行走过程的关键要素是一组页结构缓存[45]，它缓存了页表结构中的热点条目。对于 4 级页表，我们有最低有效的十二位（11:0）用于页面偏移（未转换），并且页面编号的位 47:12。虽然 TLB 中的每个条目都是一个完整的单独转换，但页结构缓存仅覆盖地址的上 3 级（位 47:21）。其思想是减少在 TLB 未命中的情况下所需的加载数量。例如，如果我们在地址的第 1 级和第 2 级找到了一个转换（位 47:30），则我们只需要执行剩下的 2 个加载。

Goldencove 微体系结构有四个专用页行走器，允许它同时处理 4 个页面行走。在 TLB 未命中的情况下，这些硬件单元将向内存子系统发出所需的加载，并使用新条目填充 TLB 层次结构。由页行走器生成的页表加载可以在 L1、L2 或 L3 缓存中命中（详细信息未公开）。最后，页行走器可以预测未来的 TLB 未命中，并在未命中实际发生之前进行推测性页面行走以更新 TLB 条目。

Goldencove 的规格未公开两个 SMT 线程之间资源共享的方式。但是一般来说，为了提高这些资源的动态利用，缓存、TLB 和执行单元是完全共享的。另一方面，用于在主要流水线阶段之间分段指令的缓冲区要么被复制，要么被分区。这些缓冲区包括 IDQ、ROB、RAT、RS、LDQ 和 STQ。PRF 也是复制的。

3.9 性能监控单元

每个现代 CPU 都提供了监控性能的设施，这些设施被合并到了性能监控单元（PMU）中。该单元集成了帮助开发人员分析其应用程序性能的功能。一个现代 Intel CPU 中的 PMU 示例如图 23 所示。大多数现代 PMU 都有一组性能监控计数器（PMC），可用于收集程序执行过程中发生的各种性能事件。稍后在 Section 5.3中，我们将讨论如何使用 PMC 进行性能分析。此外，PMU 还具有其他增强性能分析的功能，如 LBR、PEBS 和 PT，Chapter 6专门讨论了这个话题。

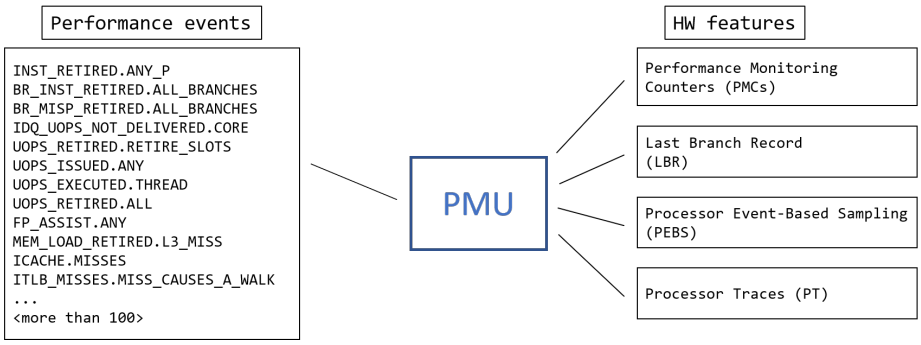


Figure 23: 现代 Intel CPU 的性能监控单元

随着每一代 CPU 设计的演进，它们的 PMU 也在发展。在 Linux 上，可以使用`cpuid`命令确定 CPU 中 PMU 的版本，如 Listing 3.9所示。类似的信息可以通过检查`dmesg`命令的输出从内核消息缓冲区中提取。每个 Intel PMU 版本的特性，以及与上一个版本的变化，可以在 [Intel, 2023b, Volume 3B, Chapter 20] 中找到。

查询 PMU 的列表:

```
$ cpuid
...
Architecture Performance Monitoring Features (0xa/eax):
      version ID                               = 0x4 (4)
      number of counters per logical processor = 0x4 (4)
      bit width of counter                     = 0x30 (48)
```

[45] AMD 的等效物称为 Page Walk Caches。

```
...
Architecture Performance Monitoring Features (0xa/edx):
      number of fixed counters    = 0x3 (3)
      bit width of fixed counters = 0x30 (48)
...
```

3.9.1 性能监控计数器

如果我们想象一下对处理器的简化视图，它可能看起来像图 24 所示的样子。正如我们在本章前面讨论过的，现代 CPU 具有缓存、分支预测器、执行流水线和其他单元。当连接到多个单元时，PMC 可以从中收集有趣的统计信息。例如，它可以计算经过了多少个时钟周期，执行了多少条指令，在此期间发生了多少缓存失效或分支预测错误等性能事件。

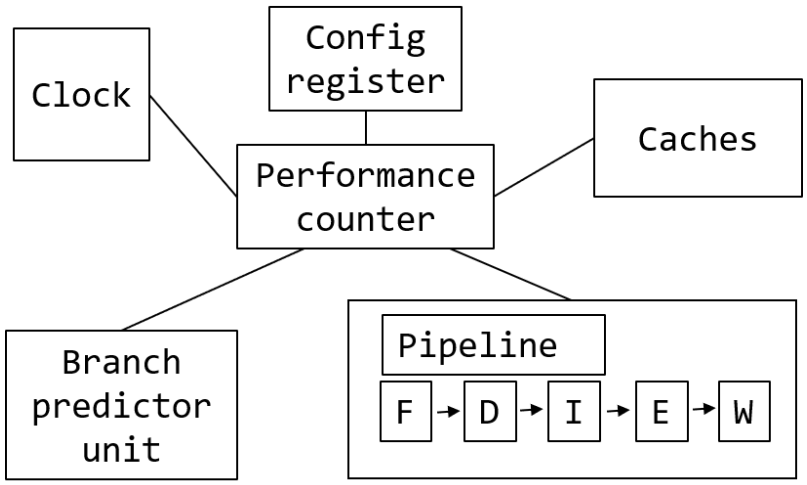


Figure 24: 带有性能监控计数器的 CPU 的简化视图。

通常，PMC 的宽度为 48 位，这使得分析工具能够在不中断程序执行的情况下运行很长时间[46]。性能计数器是作为模型特定寄存器（MSR）实现的硬件寄存器。这意味着计数器的数量和它们的宽度会因型号而异，您不能依赖于在您的 CPU 中的相同数量的计数器。您应该始终首先查询它，例如使用`cpuid`等工具。PMC 可以通过`RDMSR`和`WRMSR`指令访问，这些指令只能在内核空间执行。幸运的是，只有当您是性能分析工具的开发人员时，才需要关注这一点，例如 Linux perf 或 Intel Vtune 分析器。这些工具处理了编程 PMC 的所有复杂性。

当工程师分析他们的应用程序时，他们通常会收集已执行的指令数和经过的周期数。这就是为什么一些 PMU 具有专用的 PMC 用于收集这些事件的原因。固定计数器始终在 CPU 核心内部测量相同的事物。对于可编程计数器，用户可以选择要测量的内容。

例如，在 Intel Skylake 架构（PMU 版本 4，参见 Listing 3.9），每个物理核心有三个固定计数器和八个可编程计数器。这三个固定计数器分别设置为计算核心时钟、参考时钟和已退役 (retired) 指令（有关这些指标的更多详细信息，请参见 Chapter 4）。AMD Zen4 和 ARM Neoverse V1 核心每个处理器核心支持 6 个可编程性能监控计数器，没有固定计数器。

PMU 提供了 100 多种可用于监控的事件并不罕见。图 23 仅显示了现代 Intel CPU 上供监控的性能事件的一小部分。不难注意到可用 PMC 的数量远远小于性能事件的数量。无法同时计算所有事件，但是分析工具通过在程序执行期间在性能事件组之间进行复用来解决此问题（参见 Section 5.3.3）。

[46] 当 PMC 的值溢出时，必须中断程序的执行。然后，软件应该保存溢出的事实。我们稍后将详细讨论它。

- 对于 Intel CPU，可以在 [Intel, 2023b, Volume 3B, Chapter 20] 中找到性能事件的完整列表，或者在perfmon-events.intel.com上找到。
- AMD 并不为每个 AMD 处理器发布性能监控事件的列表。感兴趣的读者可以在 Linux perf 源代码中找到一些信息代码[47]。此外，您可以使用 AMD uProf 命令行工具列出可用于监控的性能事件。有关 AMD 性能计数器的一般信息，请参见 [AMD, 2023, 13.2 Performance Monitoring Counters]。
- 对于 ARM 芯片，性能事件没有如此明确定义。供应商按照 ARM 架构实现核心，但性能事件的含义和支持的事件在很大程度上变化。对于由 ARM 自己设计的 ARM Neoverse V1 处理器，性能事件的列表可以在 [Arm, 2022b] 中找到。

问题和练习

1. 描述流水线处理、乱序执行和投机执行。
2. 寄存器重命名如何加速执行?
3. 描述空间局部性和时间局部性。
4. 在大多数现代处理器中，缓存行的大小是多少?
5. 构成 CPU 前端和后端的组件是什么?
6. 4 级页表的组织结构是什么？什么是页错误?
7. x86 和 ARM 架构中的默认页面大小是多少?
8. TLB（转译后备缓冲区）扮演了什么角色?

AI 回答：

1. **流水线处理**：流水线处理是一种 CPU 设计，它允许多个指令在不同的处理阶段并行执行。这种设计将指令执行分解成一系列步骤，如取指、解码、执行、存储访问和写回。每个步骤由专门的硬件单元执行，使得在一个指令完成其当前阶段的同时，下一个指令可以进入流水线的下一个阶段。这样可以提高 CPU 的吞吐量，因为它减少了指令之间的空闲时间。

2. **乱序执行**：乱序执行（Out-of-Order Execution）是一种 CPU 技术，它允许指令不按照它们在程序中的原始顺序执行。CPU 通过动态调度和寄存器重命名等技术，根据指令之间的依赖关系来决定执行顺序，从而最大化执行单元的利用率，减少等待时间，提高性能。

3. **投机执行**：投机执行是一种 CPU 技术，它基于对程序执行路径的预测来提前执行指令。例如，在分支指令（如 if 语句）的情况下，CPU 可能会预测分支的走向，并提前执行预测路径上的指令。如果预测正确，可以节省等待分支结果的时间；如果预测错误，则需要撤销或回滚错误预测的结果。

4. **寄存器重命名**：寄存器重命名是一种优化技术，它允许 CPU 为逻辑寄存器分配多个物理寄存器。这样，即使多个指令引用同一个逻辑寄存器，它们也可以在不同的物理寄存器中有不同的值，从而避免了数据冲突。这种技术通过减少数据冲突和提高指令级并行性来加速执行。

5. **空间局部性和时间局部性**：

   - **空间局部性**：指程序在短时间内倾向于访问相邻的内存地址。这种特性可以通过缓存来利用，因为缓存会存储最近访问的数据块，使得相邻地址的访问可以快速得到满足。
   - **时间局部性**：指程序在短时间内可能会多次访问同一个内存地址。这种特性意味着一旦某个数据被加载到缓存中，它很可能在不久的将来再次被访问，从而减少了对主内存的访问次数。

6. **缓存行的大小**：在大多数现代处理器中，缓存行的大小通常是 64 字节。这是缓存中存储数据的最小单位，用于减少内存访问延迟。

7. **CPU 前端和后端的组件**：

[47] AMD 核心的 Linux 源代码 - https://github.com/torvalds/linux/blob/master/arch/x86/events/amd/core.c

- **前端**：包括指令预取、指令解码和执行前的准备工作。前端负责从内存中获取指令，解码它们，并将它们准备好供后端执行。
- **后端**：包括执行单元、寄存器重命名、指令调度和结果写回。后端负责实际执行指令，处理数据，并更新寄存器和内存状态。

8. **4 级页表的组织结构**：4 级页表是一种用于虚拟内存管理的数据结构，它包含四个级别的页表，每个级别包含指向下一级页表的指针。这种结构允许将虚拟地址分解成多个部分，每个部分由不同级别的页表管理，最终映射到物理地址。页错误是指 CPU 尝试访问一个虚拟地址，但该地址在物理内存中没有对应的页面，这通常会导致操作系统介入，将缺失的页面从磁盘加载到内存中。

9. **x86 和 ARM 架构中的默认页面大小**：在 x86 架构中，默认页面大小通常是 4KB。ARM 架构支持多种页面大小，包括 4KB 和 16KB，具体取决于具体的处理器实现。

10. **TLB（转译后备缓冲区）的角色**：TLB 是一种缓存，用于存储虚拟地址到物理地址的映射。它加速了虚拟内存访问，因为 CPU 可以直接从 TLB 中获取这些映射，而不需要访问完整的页表。当 TLB 未命中时，CPU 必须执行更耗时的页表查找过程。

章节总结

- 指令集架构（ISA）是软件和硬件之间的基本契约。ISA 是计算机的抽象模型，定义了可用操作和数据类型、寄存器集、内存寻址等内容。你可以以许多不同的方式实现特定的 ISA。例如，你可以设计一个“小”核心，重视功耗效率，也可以设计一个“大”核心，追求高性能。
- 实现细节封装在术语“CPU 微体系结构”中。这个主题已经被成千上万的计算机科学家研究了很长时间。多年来，许多聪明的想法在大众市场的 CPU 中得到了实现。最值得注意的是流水线处理、乱序执行、超标量引擎、投机执行和 SIMD 处理器。所有这些技术都有助于利用指令级并行性（ILP）并提高单线程性能。
- 与单线程性能并行的是，硬件设计者开始推动多线程性能。绝大多数面向客户端的现代设备都有一个包含多个核心的处理器。一些处理器通过同时多线程（SMT）技术将可观察的 CPU 核心数量翻倍。SMT 允许多个软件线程同时在同一个物理核心上运行，共享资源。在这个方向上的一种更近期的技术被称为“混合”处理器，它将不同类型的核心组合在一个单一的封装中，以更好地支持各种工作负载。
- 现代计算机中的内存层次结构包括几个级别的缓存，反映了速度访问与大小之间的不同权衡。L1 缓存通常最接近核心，速度快但容量小。L3/LLC 缓存速度较慢但容量较大。DDR 是大多数平台集成的主要 DRAM 技术。DRAM 模块在排数和存储器宽度方面有所不同，这可能会对系统性能产生小影响。处理器可能具有多个内存通道，以同时访问多个 DRAM 模块。
- 虚拟内存是与 CPU 上运行的所有进程共享物理内存的机制。程序在其访问中使用虚拟地址，这些地址被转换为物理地址。内存空间分割为页面。x86 的默认页面大小为 4KB，ARM 为 16KB。只有页面地址会被转换，页面内的偏移量保持原样。操作系统将转换保留在页表中，页表实现为基数树。有硬件功能可提高地址转换的性能：主要是转换后备缓冲区（TLB）和硬件页行走器。此外，开发人员可以利用巨大页面在某些情况下减轻地址转换的成本（见 Section 8.4）。
- 我们查看了英特尔最近的 GoldenCove 微体系结构的设计。逻辑上，核心分为前端和后端。前端包括分支预测单元（BPU）、L1-I 缓存、指令提取和解码逻辑以及 IDQ，它将指令提供给 CPU 后端。后端包括乱序执行引擎、执行单元、加载存储单元、L1-D 缓存和 TLB 层次结构。
- 现代处理器具有性能监视功能，这些功能封装在性能监视单元（PMU）中。该单元围绕性能监视计数器（PMC）的概念构建，使得可以观察程序运行时发生的特定事件，例如缓存未命中和分支预测失败。